# Deep Learning Exercise 1

Ron Hafzadi
Lior Kotlar

April 30th, 2025

# Problem

You will find the training data at the course's moodle page. Split it to train and test sets randomly at a 1:9 ratio. We'd like to set up a small multi-layered perceptron (MLP) network to accept this data and output the proper prediction (detect / not detect).

# Questions

## a

**How would you represent these 9-mers of amino acids (a vocabulary of 20 amino acids)? How would you represent the associate alleles? Explain your reasoning?**

We decided to represent these 9-mers of amino acids as a matrix of size $20 \times 9$, where each column corresponds to one of the 9 amino acid positions, and each column is a one-hot vector of length 20 representing one of the 20 amino acids. For implementation, we flatten this matrix into a 180-dimensional vector, allowing it to be fed directly into the MLP.

As for the alleles, we represent each allele as a separate one-hot vector. If there are 6 unique alleles, then each allele is represented as a vector of length 6 with a single 1 at the index corresponding to that allele.

If an antigen is positive for a given allele, we place a 1 in the corresponding index. If an antigen is not positive for any allele, the label vector contains all zeros.

## b

**What will the network's input dimension be under this representation choice? Implement an MLP that keeps this dimension for 2 inner layers, and plot the resulting train and test losses. Does this dimension pose a training problem?**

The network's input dimension under this representation is 180, as each 9-mer antigen sequence is encoded as a flattened $20 \times 9$ one-hot matrix (i.e., 20 features for each of the 9 positions, resulting in 180 total input features).

We implemented a Multi-Layer Perceptron (MLP) with the following architecture:

- **Input layer:** 180-dimensional input vector
- **Hidden layer 1:** 180 units with ReLU activation
- **Hidden layer 2:** 180 units with ReLU activation
- **Output layer:** 6 units representing the alleles

We used the following hyper parameters:

- THRESHOLD = 0.3
- LR = 0.01
- N-EPOCS = 30
- BATCH-SIZE = 16
- FALSE-NEG-WEIGHT = 3

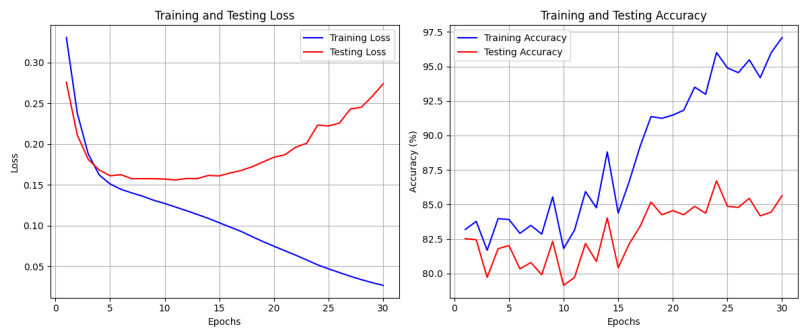
We trained the network and plotted both the training and test loss curves over epochs.

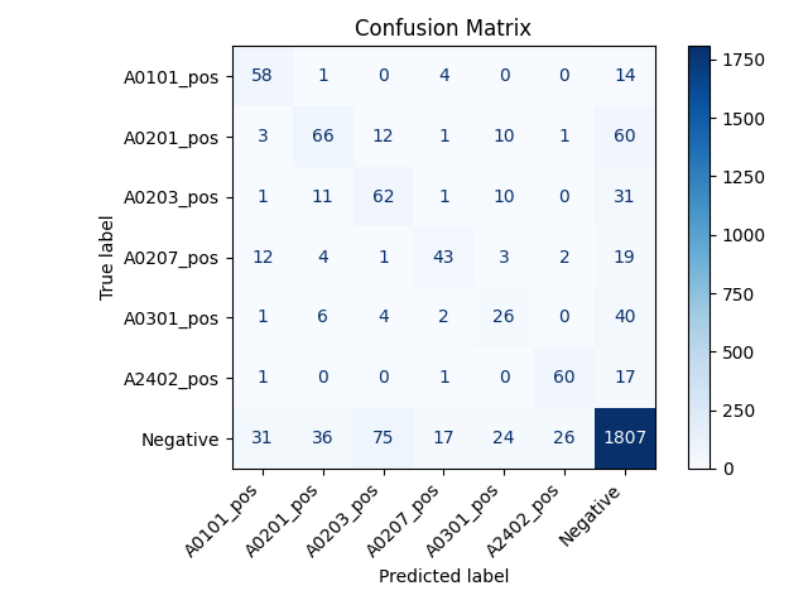
After training the network, we tested it by applying a threshold of 0.3 on the sigmoid outputs. We hypothesized that negative examples would produce relatively uniform activations across all alleles. Therefore, if none of the output values exceeded the threshold, we classified the antigen as negative. Conversely, if at least one allele's output surpassed the threshold, we considered the antigen as positively associated with that allele, therefore decided to return detected.

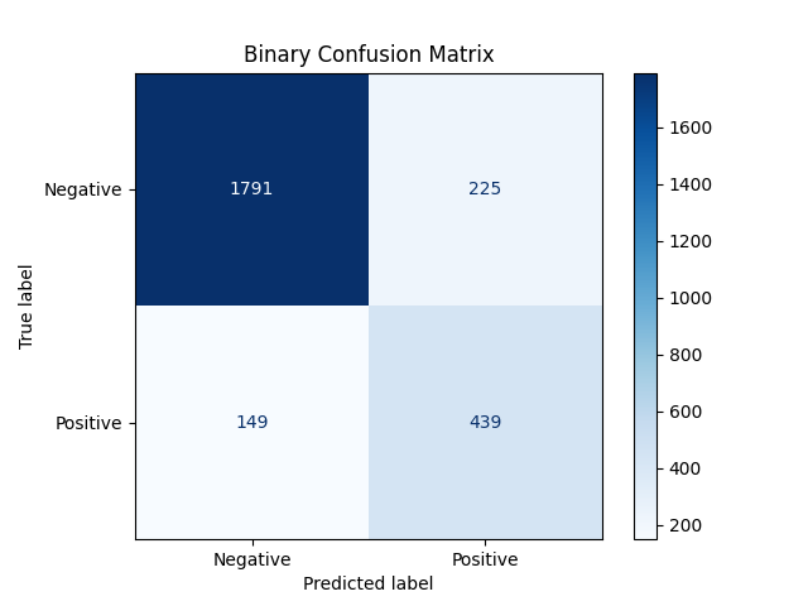
```
def decide(network_output, threshold):
    probabilities = torch.sigmoid(network_output)
    detection = torch.max(probabilities, dim=1)[0] > threshold
    return detection
```

Empirically, we found that the input dimensionality (180) is manageable. The model trains stably, and both train and test losses converge.

**On the test set, we have reached an accuracy of** 85.64**, with precision of** 0.6611 **and recall of** 0.7466**.**




**c**

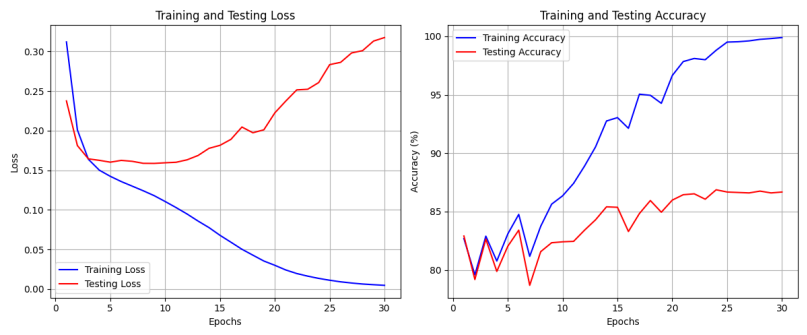
**Describe a network architecture that avoids the problems seen, explain all your design choices. Plot the train / tests losses. Include the plots obtained in your report.**

We implemented a Multi-Layer Perceptron (MLP) with the following architecture:

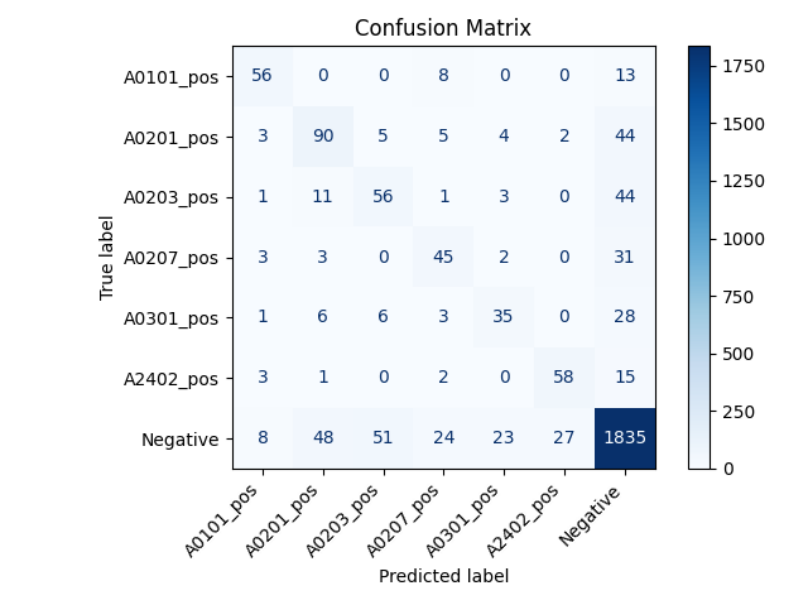
- **Input layer:** 180-dimensional input vector
- **Hidden layer 1:** 1024 units with ReLU activation
- **Hidden layer 2:** 1024 units with ReLU activation
- **Output layer:** 6 units representing the alleles

In order to methodically compare between the two networks, we pumped the hidden layers sizes to 1024. we kept all other Hyper-params the same.

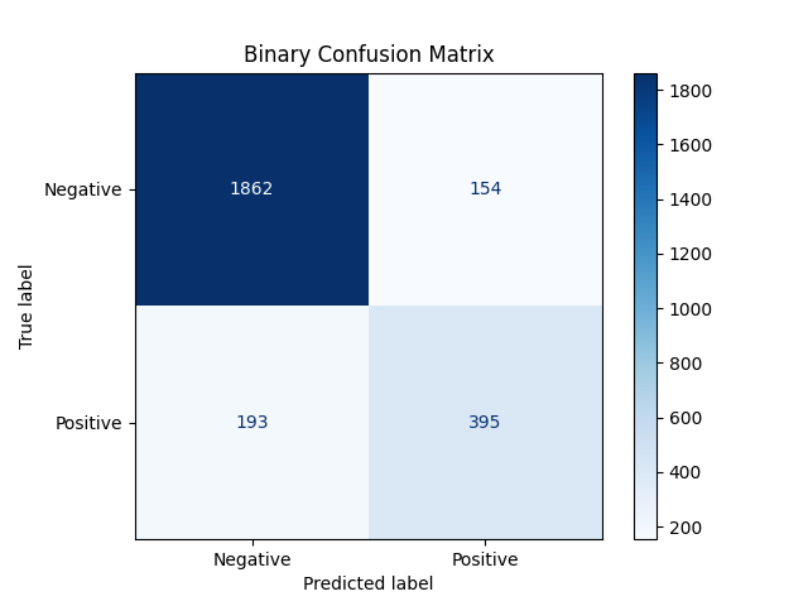
We trained the network and plotted both the training and test loss curves over epochs.




**On the test set, we have reached an accuracy of** 86.87**, with precision of** 0.7195 **and recall of** 0.6718**.**



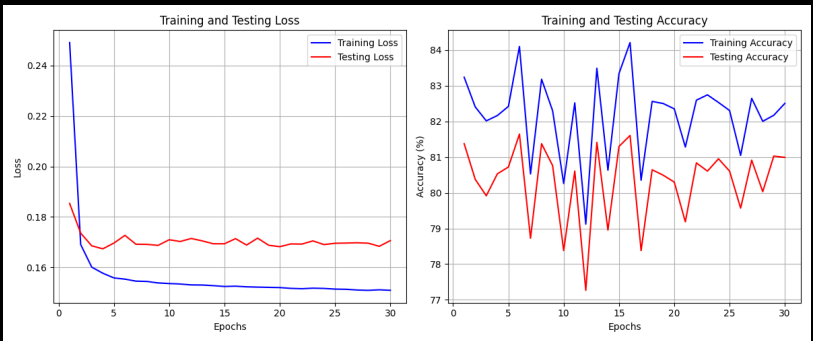

**When comparing the results of the model presented in section C and the model presented in section B, a worse test loss plot is seen in the C model graph. This can be attributed to overfitting. The overfitting arises from a more complex model that performs well on the training set but worse on the testing set.**




## d

**Remove the non-linear operators from your network, and report the results obtained. How do the results compare?**

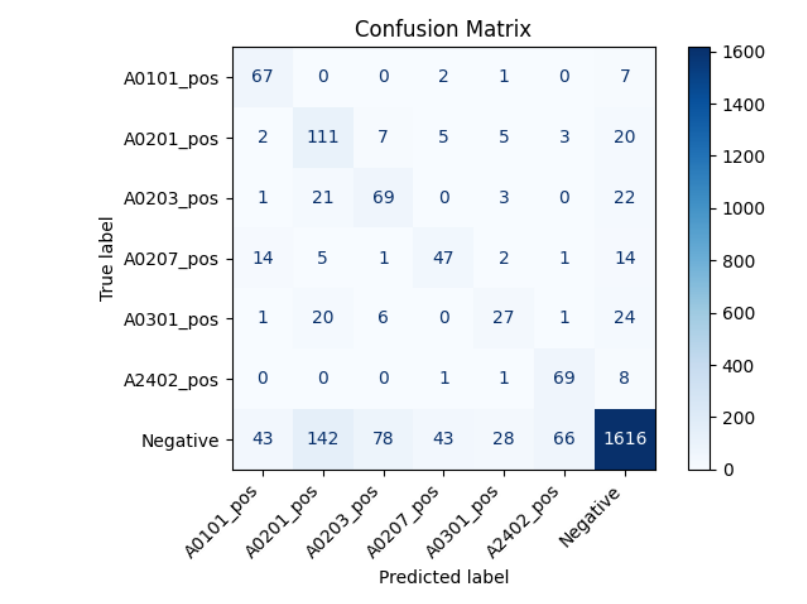
In this section, we took the same network of network c, and removed the relu activations.
We kept all other Hyper-params the same.
We trained the network and plotted both the training and test loss curves over epochs.

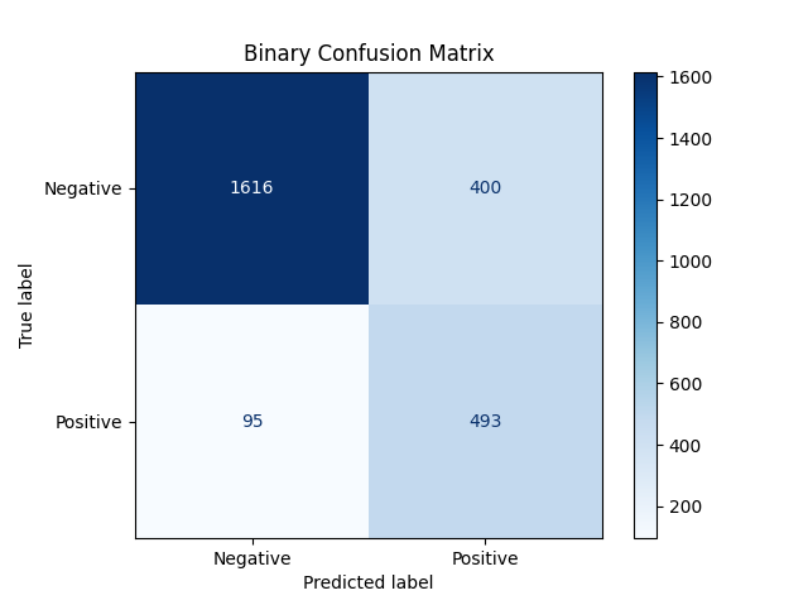



**On the test set, we have reached an accuracy of** 80.99**, with precision of** 0.5521 **and recall of** 0.8384**.**

### Comparing the Linear Model to the Non-Linear Model

the linear model converged quickly but to a high loss, indicating underfitting due to limited capacity - it could not capture the complexity of the data. The non-linear model achieved a much lower training loss, showing better learning ability but its test loss increased after initial improvement indicating overfitting.




**e**

We used all 3 of our models to predict the detection of 9-amino-acids peptides from the Spike protein of the SARS-CoV-2.

**MLP-B**

| Rank | Peptide | Allele | Score |
|---|---|---|---|
| 1 | MYICGDSTE | A2402_pos | 12.9309 |
| 2 | KYNENGTIT | A2402_pos | 11.7229 |
| 3 | NYKLPDDFT | A2402_pos | 11.0774 |

**MLP-C**

| Rank | Peptide | Allele | Score |
|---|---|---|---|
| 1 | KYNENGTIT | A2402_pos | 11.5826 |
| 2 | NYKLPDDFT | A2402_pos | 10.6993 |
| 3 | KWPWYIWLG | A2402_pos | 8.6088 |

**MLP-D**

| | | | |
|---|---|---|---|
| 1 | MYICGDSTE | A2402_pos | 6.7252 |
| 2 | YNENGTITD | A0101_pos | 6.5486 |
| 3 | NYKLPDDFT | A2402_pos | 6.1772 |

## Preprocessing and Hyperparameters

Our first step was to remove duplicates, samples that were both in the negative and positive alleles, and samples that were in more than 1 positive allele were all removed to not harm the training process.

As the number of examples, and more importantly positive exmaples was sparse after cleaning, we decided to change our loss function to punish more for false negatives, which suits both the data distribution, and the fact that we prefer false positive than false negative when testing for illnesses. Hence, we decided that we punish times 3 when doing a false negative.

To identify the optimal hyperparameters, we performed an exhaustive search over a range of reasonable values for each model variant. Specifically, we varied the number of epochs, threshold, weights of positives, and batch size.

```
for epochs in (20, 30, 40, 50):
    for letter in ('B', 'C', 'D'):
        for threshold in (0.2, 0.3, 0.4, 0.5, 0.6):
            for weights in (1, 2, 3):
                for batchsize in (32, 16, 8):
                    if letter == 'B':
                        model = MLP_B()
                    elif letter == 'C':
                        model = MLP_C()
                    else:
                        model = MLP_D()
                    train_loader, test_loader = get_train_test(
                        alleles_dict, negative_antigens, weights, 0.1, batchsize)
                    model = train_model(model, train_loader, epochs)
                    test_model(model, test_loader, batchsize, threshold, weights, epochs)
```

Training with weights resulted in lower accuracy on the test set. After reflecting on the objectives of the task, we determined that accuracy alone is not the most appropriate metric. Therefore, we focused on optimizing recall, which better capture the model's ability to correctly identify true positives without being misled by the abundance of negative examples. Consequently, we selected the Hyper-params that maximized recall, rather than those that yielded the highest overall accuracy.

# Home Assignment 1

## 1.(a) Linearity

$f: \mathbb{R}^n \to \mathbb{R}^m$, $f$ linear. from linearity of $f$, denote $f(x) = Ax$

$g: \mathbb{R}^m \to \mathbb{R}^l$, $g$ linear. similarly to $f$, $g(x) = Bx$

n.t.s $h(x) = g(f(x))$ is linear. $(h(x) = B(Ax) = BAx)$

> Properties of linearity: Additivity: $f(x+y) = f(x) + f(y)$,
> Homogeneity: $f(\alpha \cdot x) = \alpha \cdot f(x)$

Proof: Additivity: $h(x+y) = BA(x+y) = BAx + BAy = \underline{h(x) + h(y)}$

Homogeneity: $h(\alpha x) = BA(\alpha x) = \alpha BAx = \underline{\alpha \cdot h(x)}$

$\Rightarrow$ $h$ is linear

---

## 1.(b) Affinity

$f: \mathbb{R}^n \to \mathbb{R}^m$, $f$ affine. from affinity of $f$, denote $f(x) = Ax + a$

$g: \mathbb{R}^m \to \mathbb{R}^l$, $g$ affine. similarly to $f$ $g(x) = Bx + b$ — constants

n.t.s $h(x) = g(f(x))$ is affine. $h(x) = B(Ax+a) + b = BAx + Ba + b$

Proof: $h(x) = BAx + Ba + b$. Denote $BA = C$, $Ba + b = d$

$\Rightarrow h(x) = Cx + d \Rightarrow C$ is linear, and $d$ is constant w.r.t $x$

$\Rightarrow \underline{h \text{ is affine}}$

# 2. Gradient Descent

a. obj function - $\min\left[f(x,y)=10\cdot(x-1)^2+\frac{(y+1)^2}{10}\right]$.

$\frac{\partial f}{\partial x}=10\cdot 2(x-1)=20(x-1)$, $\frac{\partial f}{\partial y}=\frac{1}{10}\cdot 2\cdot(y+1)=\frac{1}{5}(y+1)$

$\eta=LR \Rightarrow x_{k+1}=x_k-\eta\cdot\frac{\partial f}{\partial x}\Rightarrow x_{k+1}=x_k-\eta\cdot 20(x_k-1)=\underbrace{x_k(1-20\eta)}_{r_x}+\eta\cdot 20$

$y:=y-\eta\cdot\frac{\partial f}{\partial y}\Rightarrow y_{k+1}=y_k-\eta\cdot\frac{1}{5}(y+1)=y_k\underbrace{(1-\eta\cdot\frac{1}{5})}_{r_y}-\eta\frac{1}{5}$

---

b. maximum learning rate

first let's compute the hessian:

$\frac{\partial^2 f}{\partial x^2}=20$, $\frac{\partial^2 f}{\partial y^2}=\frac{1}{5}$, $\frac{\partial^2 f}{\partial x\partial y}=0\Rightarrow H=\begin{pmatrix}20 & 0\\ 0 & \frac{1}{5}\end{pmatrix}$

eigenvalues of H are $\lambda_1=20$, $\lambda_2=\frac{1}{5}\Rightarrow\lambda_{max}=20$

$\eta_{max}=\frac{2}{\lambda_{max}}=\frac{2}{20}=\frac{1}{10}$. for any $\eta>\frac{1}{10}$ gradient descent will not converge

convergence guaranteed iff $|r_x|,|r_y|<1\Rightarrow|r_x|<0\Rightarrow|1-20\eta|<1$

$\Rightarrow -1<1-20\eta<1\Rightarrow -2<-20\eta<0\Rightarrow 0<\eta<\frac{1}{10}\Rightarrow$ max $LR=\frac{1}{10}$

---

C. Becase took $\lambda_{max}=20$ which came from $\frac{\partial^2 f}{\partial x^2}$, so the GD in the direction of x should be more cautious. so the '10' in the objective function together with the '2' in the power, determine the max LR.

---

d. y takes the longest. that's determined by the smallest 2nd derivative. the convergence depends on $r_y=1-\eta\cdot\frac{1}{5}$ which is closer to 1 than $r_x=1-20\eta$. the closer r is to 1 the slower the convergence.

3.

- pred_deg := pred_deg % 360
- true_deg := true_deg % 360
- diff := |true_deg − pred_deg|
- loss := min(diff, 360-diff)
- return loss

---

# 4. Chain Rule

a) $\frac{\partial}{\partial x} f(x+y, 2x, z)$. Denote $u(x,y)=x+y$, $v(x)=2x$, $w(z)=z$

$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u}\cdot\frac{\partial u}{\partial x} + \frac{\partial f}{\partial v}\cdot\frac{\partial v}{\partial x} + \frac{\partial f}{\partial w}\cdot\frac{\partial w}{\partial x}$. $\frac{\partial u}{\partial x}=1$, $\frac{\partial v}{\partial x}=2$, $\frac{\partial w}{\partial x}=0$

$\Rightarrow \boxed{\frac{\partial f}{\partial u} + 2\cdot\frac{\partial f}{\partial v}}$

---

b) $F(x) = f_1(f_2(\dots f_n(x)\dots))$. $\frac{\partial F}{\partial x} = \frac{\partial f_1}{\partial f_2}\cdot\frac{\partial f_2}{\partial f_3}\cdot\frac{\partial f_3}{\partial f_4}\cdot\dots\cdot\frac{\partial f_n}{\partial x}$

$\frac{\partial F}{\partial x} = \boxed{f_1'(f_2(\dots))\cdot f_2'(f_3(\dots))\cdots f_n'(x)}$

---

c) $F(x) = f_1(x, f_2(x, \dots f_{n-1}(x, f_n(x))\dots))$

$\frac{\partial f_k}{\partial x} = \frac{\partial f_k}{\partial x} + \frac{\partial f_k}{\partial f_{k+1}}\cdot\frac{\partial f_{k+1}}{\partial x} \Longrightarrow \boxed{\frac{\partial F}{\partial x} = \frac{\partial f_1}{\partial x} + \frac{\partial f_1}{\partial f_2}\left(\frac{\partial f_2}{\partial x} + \frac{\partial f_2}{\partial f_3}\left(\dots\frac{\partial f_{n-1}}{\partial x} + \frac{\partial f_{n-1}}{\partial f_n}\cdot f_n'(x)\dots\right)\right)}$

---

d) $F(x) = f(x+g(x+h(x)))$ $u(x)=x+g(x+h(x))$, $v(x)=x+h(x)$

$F(x)=f(u(x)) \Rightarrow \frac{\partial F}{\partial x} = \frac{\partial f}{\partial u}\cdot\frac{\partial u}{\partial x} = \frac{\partial f}{\partial u}\left(1+\frac{\partial g}{\partial x}\right)$, $\frac{\partial g}{\partial x} = \frac{\partial g}{\partial v}\cdot\frac{\partial v}{\partial x} = \frac{\partial g}{\partial v}\cdot\left(1+\frac{\partial h}{\partial x}\right)$

$\boxed{\frac{\partial F}{\partial x} = \frac{\partial f}{\partial u}\left(1+\frac{\partial g}{\partial v}\left(1+\frac{\partial h}{\partial x}\right)\right)}$